QUICKBITE

FOOD ORDERING WEBSITE

PROJECT REPORT

ACADEMIC PROJECT REPORT

Prepared for academic project submission

Submitted by	Akash A N
Register / Roll No.	225009
Course / Programme	B.TECH
Department	Computer Science Engineering
Institution	Sahrdaya College Of Engineering And Technology
Project Guide	Dr.Anil Antony
Academic Year	2026-27

Live website: https://akash225009-cloud.github.io/QuickBite/

CERTIFICATE

This is to certify that the project report entitled “QuickBite - Food Ordering Website” is submitted by [Student Name], Register Number [Register Number], of [Department / Programme], [Institution Name], as part of the academic requirements for [Course / Degree].

The work described in this report relates to the design, implementation and deployment of a web-based food catalogue and ordering interface using React, Node.js, Express and MongoDB. The report has been prepared for review and submission; project-specific institutional statements should be approved and completed by the student and project guide.

DECLARATION

I, [Student Name], hereby declare that this report titled “QuickBite - Food Ordering Website” is a description of the project work completed for academic purposes. The technical description is based on the implemented project and the deployment steps followed. Any external material used in the final submitted version should be properly acknowledged, and the details on this page should be reviewed and completed before submission.

ACKNOWLEDGEMENT

I would like to express my gratitude to [Project Guide Name] for guidance and feedback during the development of this project. I also thank the faculty and department of [Institution Name] for providing the opportunity to work on a practical web application. I acknowledge the maintainers of the open-source tools used in the project, including React, Vite, Node.js, Express and Mongoose, as well as the hosting and database platforms used to publish and operate the application.

ABSTRACT

QuickBite is a web-based food ordering project intended to present food items through a browser-based user interface. The system is implemented as a separate React frontend and Node.js/Express backend, with MongoDB Atlas used to store food catalogue records. The frontend is built with Vite and published through GitHub Pages, while the backend is deployed on Render. Communication between the frontend and backend takes place through HTTP API requests.

The backend defines a Food data model with fields for name, price, category, description, image and availability. It exposes a root route to confirm that the service is running, a GET /foods route to retrieve the menu, and a POST /add-sample-foods route to populate initial example data when the collection is empty. During deployment verification, the backend successfully connected to MongoDB Atlas and the /foods endpoint returned four sample food records: Chicken Burger, Margherita Pizza, French Fries and Chicken Biryani.

The project demonstrates full-stack separation, REST-style API usage, persistent data storage for the food catalogue, environment-based configuration and static frontend deployment. The scope of the backend code confirmed for this report is focused on menu retrieval and sample-data initialization. Persistent order creation, payment processing, authentication and delivery tracking are not described as completed backend functions because no corresponding API routes were present in the supplied server implementation.

Keywords

Food Ordering Website, React, Vite, Node.js, Express, MongoDB Atlas, Mongoose, REST API, GitHub Pages, Render

TABLE OF CONTENTS

Section	Contents
Certificate	Project certification template
Declaration	Student declaration template
Acknowledgement	Acknowledgement
Abstract	Project summary and keywords
Chapter 1 - Introduction	Background, problem, objectives and scope
Chapter 2 - System Study	Existing approach, proposed system and feasibility
Chapter 3 - Requirements	Functional and non-functional requirements
Chapter 4 - Design	Architecture, modules, data model and workflow
Chapter 5 - Implementation	Technology stack, project structure and API
Chapter 6 - Testing	Test cases and observed results
Chapter 7 - Deployment	Local and cloud deployment process
Chapter 8 - Conclusion	Limitations and future improvements
References	Official technology documentation

CHAPTER 1 - INTRODUCTION

1.1 Background

Food ordering websites provide a digital way to explore food items and view key details such as price, category and availability. A small web application is a useful learning project because it connects a user interface, a server-side API and a database. QuickBite follows this pattern by separating the React frontend from the Express backend and storing catalogue data in MongoDB.

1.2 Problem Statement

A static menu alone cannot conveniently provide centrally stored food data to multiple browser sessions. A small online system therefore needs a persistent catalogue, a server endpoint to retrieve that catalogue, and a browser interface that requests the data through an API. Localhost addresses also cannot be used by external visitors, so production deployment requires a public backend URL and suitable cross-origin access.

1.3 Aim

To design, implement and deploy a web-based food ordering interface backed by a server-side food catalogue API and a cloud-hosted database.

1.4 Objectives

- Build a browser-based frontend with React and Vite.
- Create an Express API that serves food catalogue records.
- Define and store food data using Mongoose and MongoDB Atlas.
- Provide an endpoint for one-time insertion of sample menu records when the collection is empty.
- Deploy the frontend on GitHub Pages and the backend on Render.
- Configure the frontend to call the deployed backend rather than localhost.
- Document setup, deployment, testing and known limitations.

1.5 Scope

The confirmed backend scope includes service availability, food catalogue retrieval and sample menu initialization. The frontend is published as a static React application. The project illustrates a foundation for an online food ordering system; order persistence, payment integration, customer accounts and delivery workflows would require additional backend models, endpoints and security controls.

CHAPTER 2 - SYSTEM STUDY

2.1 Existing Approach

A simple menu maintained only in client-side code can be easy to demonstrate but is difficult to update centrally and does not provide a persistent shared data source. A locally running API is also inaccessible to users outside the development computer unless it is deployed to a public host.

2.2 Proposed System

QuickBite separates responsibilities into three layers: the React frontend, the Express API and the MongoDB Atlas database. The browser sends an HTTP request to the deployed backend; the backend queries MongoDB via Mongoose and returns JSON. GitHub Pages serves the frontend bundle, and Render runs the Node.js process.

2.3 Feasibility

Type	Assessment
Technical	The stack uses widely used web technologies and a simple REST endpoint for menu retrieval. The application has been deployed with GitHub Pages, Render and MongoDB Atlas.
Operational	Users can open the published website in a modern browser. Menu data is obtained through the hosted API.
Economic	The project can be demonstrated using the free or entry-level tiers of the selected services, subject to platform limits and current plan policies.
Maintenance	Frontend, backend and database configuration are separated, making it easier to update each component independently.

2.4 Intended Users

The primary audience is a visitor exploring the food menu. The repository and deployment process are also intended to be used by a student developer or evaluator to inspect and demonstrate the implementation. Administrative and restaurant-owner roles are not claimed as implemented unless separate code is added.

CHAPTER 3 - REQUIREMENTS

3.1 Hardware Requirements

- Computer capable of running a current Node.js release and a modern web browser.
- Internet connection for GitHub, Render and MongoDB Atlas.
- No dedicated server hardware is required for the deployed version.

3.2 Software Requirements

Component	Technology / Requirement	Purpose
Frontend	React 19.3.1, Vite 8.3.4	Build and serve the browser interface.
Backend	Node.js, Express 5.3.0	HTTP server and API routes.
ODM	Mongoose 9.11.4	Schema definition and MongoDB operations.
Database	MongoDB Atlas	Cloud storage for food records.
Configuration	dotenv 18.0.9	Load local environment values from .env.
Cross-origin access	cors 2.8.6	Allow frontend requests to the API.
Frontend hosting	GitHub Pages + GitHub Actions	Publish built static assets.
Backend hosting	Render Web Service	Run the Node.js server.

3.3 Functional Requirements

- **FR-01: Service status.** The API root endpoint responds with a text message confirming that QuickBite is running.
- **FR-02: Read food catalogue.** The GET /foods endpoint returns food documents stored in MongoDB.
- **FR-03: Initialize sample data.** The POST /add-sample-foods endpoint inserts a predefined sample set only when the Food collection is empty; otherwise it reports that records already exist.
- **FR-04: Display the hosted frontend.** The static React application is accessible through its GitHub Pages URL.
- **FR-05: Connect to the online API.** The frontend uses the public Render service URL rather than a localhost address in production.

3.4 Non-Functional Requirements

- **Usability:** Food names, categories, descriptions and prices should be presented clearly in the interface.
- **Availability:** The hosted frontend and backend should be reachable when their respective services are healthy; free-tier cold starts may add delay.
- **Maintainability:** Frontend and backend are separated into the client and server directories.
- **Configuration safety:** Database credentials must be kept in environment variables and excluded from version control.
- **Compatibility:** The frontend is a static Vite build and should be tested in supported modern browsers.

CHAPTER 4 - SYSTEM DESIGN

4.1 High-Level Architecture

```
VISITOR / BROWSER
      |
      v
React + Vite frontend (GitHub Pages)
      |  HTTP request: GET /foods
      v
Node.js + Express API (Render)
      |  Mongoose query
      v
MongoDB Atlas (Food collection)
      |
      v
JSON response returned to frontend
```

Figure 4.1. Logical architecture of the deployed QuickBite application.

4.2 Module Description

Module	Responsibilities
Frontend UI	Renders the food ordering interface and requests food data from the backend using the configured API base URL.
Express server	Creates the HTTP application, parses JSON request bodies, enables CORS and registers the API endpoints.
Food model	Defines the MongoDB document structure using a Mongoose schema.
Food API	Retrieves catalogue records and optionally creates the initial sample set.
Database layer	MongoDB Atlas stores food documents; the server connects using MONGODB_URI.
Deployment workflow	GitHub Actions builds the client directory and publishes the generated dist output to GitHub Pages.

4.3 Database Design

The current backend declares one collection-backed entity: Food. MongoDB documents use the Mongoose-generated identifier and the fields below. Mongoose also provides the document version field visible in the sample JSON (__v).

Field	Type	Meaning / Example
_id	ObjectId	Unique MongoDB document identifier.
name	String	Food name, e.g. Chicken Burger.
price	Number	Price value, e.g. 149.
category	String	Menu category, e.g. Burger or Pizza.
description	String	Short description of the food item.
image	String	Image path or URL; supplied sample records use an empty string.
available	Boolean	Whether the item is available; defaults to true.
__v	Number	Mongoose document version key.

CHAPTER 5 - IMPLEMENTATION

5.1 Technology Stack

The project uses a JavaScript full-stack architecture. React renders the UI; Vite provides the development server and production build; Express provides HTTP routing; Mongoose maps JavaScript models to MongoDB documents; MongoDB Atlas hosts persistent catalogue data.

5.2 Project Structure

```
QuickBite/
|-- .github/
|   `-- workflows/
|       `-- deploy.yml        # GitHub Pages build/deploy workflow
|-- client/
|   |-- src/
|   |   |-- App.jsx           # Main React UI and API requests
|   |   |-- App.css           # App-specific styles
|   |   |-- index.css         # Global styles
|   |   `-- main.jsx          # React entry point
|   |-- public/               # Static assets
|   |-- package.json          # Vite scripts and React dependencies
|   `-- vite.config.js        # GitHub Pages base path
|-- server/
|   |-- index.js              # Express server and food endpoints
|   |-- package.json          # Backend start script and dependencies
|   `-- .env                  # Local secrets; do not commit
`-- .gitignore
```

5.3 Backend Endpoints

Method	Path	Purpose	Observed behaviour
GET	/	Basic service check.	Returns “QuickBite backend is running!”.
GET	/foods	Read food catalogue.	Returns an array of Food documents from MongoDB.
POST	/add-sample-foods	Initial sample data.	If count is zero, inserts four sample foods; otherwise returns a message and existing count.

5.4 Sample Catalogue Data

Name	Price	Category	Description
Chicken Burger	149	Burger	Juicy chicken burger
Margherita Pizza	249	Pizza	Classic cheese pizza
French Fries	99	Snacks	Crispy golden fries
Chicken Biryani	199	Biryani	Aromatic chicken biryani

CHAPTER 6 - TESTING AND RESULTS

The following table records checks reported during the project setup. “Passed” means the test was observed in the development/deployment conversation; it is not a substitute for retesting before formal evaluation.

ID	Test	Expected result	Recorded result
T01	Start backend locally using npm start.	Server starts and connects to MongoDB.	Passed: console showed MongoDB connected and server running on port 5000.
T02	Open the deployed API root (/).	Service-running message is returned.	Passed: browser displayed “QuickBite backend is running!”.
T03	Open the deployed /foods endpoint.	Food records are returned as JSON.	Passed: four sample records were returned.
T04	Open the GitHub Pages frontend.	Frontend assets load.	Passed: user confirmed frontend appears.
T05	Connect frontend to Render URL.	Frontend retrieves menu without localhost dependency.	Passed: user later confirmed the site was working.
T06	Create and persist a customer order.	Order record is stored and retrievable.	Not verified; no order endpoint was present in the backend code shared.
T07	Payment, user login and delivery tracking.	Related workflows operate securely.	Not implemented/verified in the backend code shared.
T08	Cross-browser, accessibility and load testing.	Consistent behaviour across environments.	Not formally tested.

6.1 Deployment Outcome

The frontend was published at the GitHub Pages URL and the backend was deployed as a Render web service. The /foods request returned the sample food catalogue from MongoDB Atlas. The frontend API base URL was changed to the Render service URL to resolve the production connection error.

6.2 Acceptance Checklist

- Open the public website in an incognito/private window.
- Verify that the menu loads without a backend connection error.
- Refresh the page and confirm the food catalogue still loads.
- Open the API root and /foods endpoint directly.
- Check GitHub Actions for a successful frontend deployment.
- Check Render logs for a successful database connection.
- Do not claim order persistence, payments or authentication unless they are separately implemented and tested.

CHAPTER 7 - DEPLOYMENT AND OPERATION

7.1 Local Setup

1. Install a current supported Node.js release and npm.
2. Clone/download the QuickBite repository from GitHub.
3. In the server directory, run npm install.
4. Create server/.env with MONGODB_URI set to the MongoDB Atlas connection string.
5. Run npm start in server and verify the root endpoint at http://localhost:5000/.
6. In a second terminal, move to client and run npm install, then npm run dev.
7. For local development, configure the frontend API base URL to http://localhost:5000; for production, use the Render base URL.

7.2 Backend Deployment on Render

Setting	Value
Repository	https://github.com/akash225009-cloud/QuickBite
Root Directory	server
Build Command	npm install
Start Command	npm start
Environment	MONGODB_URI from MongoDB Atlas; Render provides PORT

7.3 Frontend Deployment on GitHub Pages

- The Vite base path is configured as /QuickBite/ for the repository name.
- The GitHub Actions workflow installs client dependencies, runs npm run build and publishes client/dist.
- In repository Settings > Pages, set the deployment source to GitHub Actions.
- Push client code changes to main; check the Actions tab until the workflow completes successfully.

7.4 Deployment URLs

Component	URL
Frontend	https://akash225009-cloud.github.io/QuickBite/
GitHub repository	https://github.com/akash225009-cloud/QuickBite
Backend root	https://quickbite-backend-9ayh.onrender.com/
Food catalogue API	https://quickbite-backend-9ayh.onrender.com/foods

CHAPTER 8 - CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Conclusion

QuickBite demonstrates how a React web interface can be linked to an Express backend and a MongoDB Atlas catalogue. The frontend and backend are hosted separately, allowing the static user interface to be delivered from GitHub Pages while the API performs database operations on Render. The observed deployment checks confirmed that the backend started, connected to MongoDB and returned the four sample food records through /foods. The published frontend was also confirmed to load after its API base URL was updated.

8.2 Current Limitations

- The backend code described in this report exposes a food catalogue read endpoint and sample-data insertion endpoint; it does not expose a persistent order creation API.
- The sample food documents use empty image values, so actual food imagery depends on additional assets or image URLs added to the data and frontend.
- No payment gateway, delivery tracking or documented authentication workflow was verified.
- CORS is configured broadly in the supplied backend; for a production-grade app it should be restricted to approved frontend origins.
- Formal browser compatibility, performance, security and accessibility testing has not been completed as part of the recorded checks.

8.3 Future Enhancements

- Add an Order schema and protected endpoints to create and retrieve orders.
- Add customer registration, login and role-based authorization with secure password handling and JWT validation.
- Build a cart and checkout flow that submits validated order details to the API.
- Integrate payment processing only through a supported provider and verify payments on the server.
- Create admin/restaurant tools to add, update and mark foods unavailable.
- Use image hosting or a managed media store for menu images.
- Restrict CORS, validate all inputs, add rate limiting and centralised error logging.
- Add automated unit and integration tests, including API and deployment smoke tests.

REFERENCES

1. React documentation: https://react.dev/
2. Vite documentation and static deployment guide: https://vite.dev/guide/
3. Node.js documentation: https://nodejs.org/docs/
4. Express documentation: https://expressjs.com/
5. Mongoose documentation: https://mongoosejs.com/docs/
6. MongoDB Atlas documentation: https://www.mongodb.com/docs/atlas/
7. GitHub Pages documentation: https://docs.github.com/pages
8. GitHub Actions documentation: https://docs.github.com/actions
9. Render documentation: https://render.com/docs